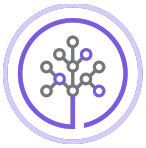
Create Django Proxy Services Of Backend APIs



Skills
Network

Estimated time needed: 120 minutes

In this lab, you will integrate previously created CarModels and CarMake models and services to manage all entities such as dealers and reviews. You are also provided with dealer and review models Mongo DB served by express API end points.

To integrate external dealer and review data, you need to call the API end points from the Django app and process the API results in Django views. Such Django views can be seen as proxy services to the end user because they fetch data from external resources per users' requests.

In this lab, you will create such Django views as proxy services.

Run the Mongo Server

The backend Mongo Express server needs to be up and running in one of the terminals in the lab environment. At this stage, the server code will have all the end points implemented already.

- Open a **New Terminal**
- Git clone your repository with all the changes you have made in the previous tasks
- Change to the database directory

```bash
cd /home/project/xrwvm-fullstack_developer_capstone/s
```

Run

- Build the nodeapp

```bash
docker build . -t nodeapp
```

Run

- Run the command given below to start the server

```bash
docker-compose up
```

Run

- Keep the server running in this terminal. You will need it for doing the rest of the lab.
- Select the **Backend** button below, copy the URL in the address bar.

Backend

▶ If the page doesn't open, select here to open the URL manually.

- Open `djangoapp/.env` and replace the `your backend url` with the URL of your backend you copied earlier in the notepad in the previous step.

> *Ensure that the `/` at the end is not copied.*

plaintext

```
backend_url =your backend url
```

Refer to the lab if required.

Environment Setup

- Open another **New Terminal**
- Run the following to set up the Django environment.

bash

```
cd /home/project/xrwvm-fullstack_developer_capstone/server

pip install virtualenv
virtualenv djangoenv
source djangoenv/bin/activate
```

Run

- Install the required packages by running the command below.

bash

```
python3 -m pip install -U -r requirements.txt
```

Run

- Run the command below to perform models migration.

bash

```
python3 manage.py makemigrations
python3 manage.py migrate
python3 manage.py runserver
```

Run

Create Function to Interact with Backend

In the previous lab, you would have created a API endpoints to fetchReviews and fetchDealers. Now implement a method to access these from the Django app.

There are many ways to make HTTP requests in Django. Here we use a very popular and easy-to-use Python library called `requests`.

- Open `djangoapp/restapis.py` and Uncomment the following import at the top of the file:

```python
import requests
```

- Add a following `get_request` method, as given below.

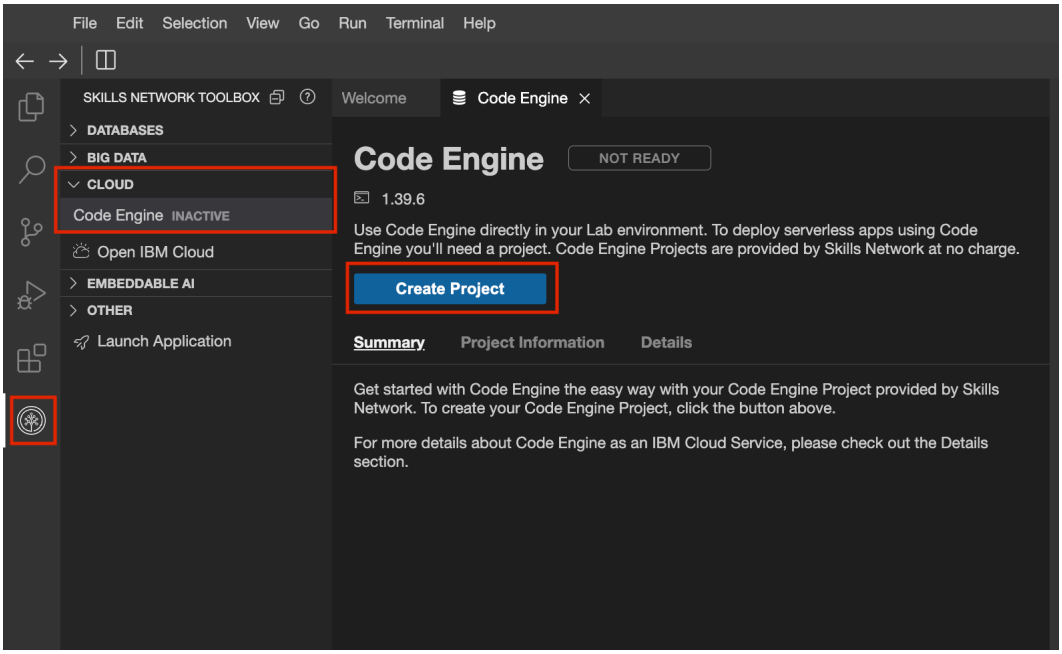
```python
def get_request(endpoint, **kwargs):
    params = ""
    if(kwargs):
        for key,value in kwargs.items():
            params=params+key+"="+value+"&"

    request_url = backend_url+endpoint+"?"+params

    print("GET from {} ".format(request_url))
    try:
        # Call get method of requests library with URL and
        response = requests.get(request_url)
        return response.json()
    except:
        # If any error occurs
        print("Network exception occurred")
```

The `get_request` method has two arguments, the endpoint to be requested, and a Python keyword arguments representing all URL parameters to be associated with the get call.

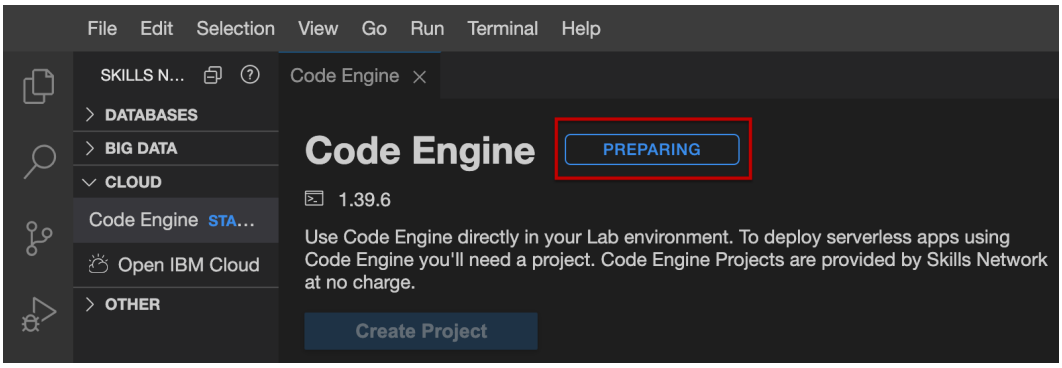
This function calls `GET` method in `requests` library with a URL and any URL parameters such as `dealerId`.

Start the Code Engine

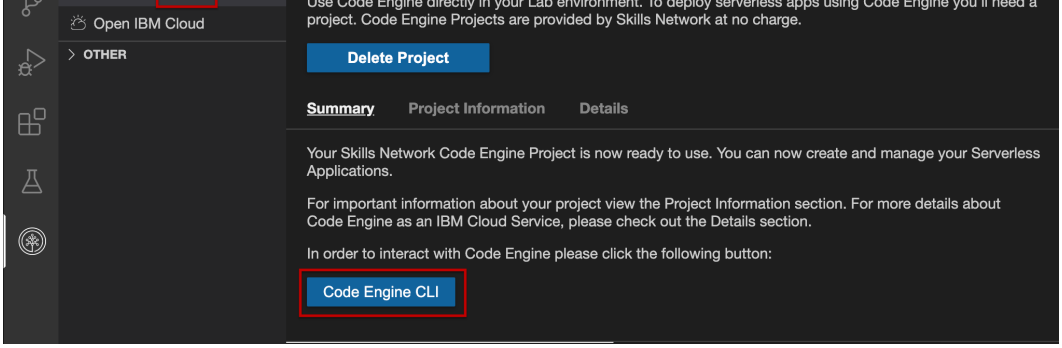
- To start **Code Engine**, select **Crate Project**



- The code engine environment takes a while to prepare. You will see the progress status being indicated in the set up panel.



- Once the code engine set up is complete, you can see that it is active. Select `Code Engine CLI` to begin the pre-configured CLI in the terminal below.



- You will observe that the pre-configured CLI statrup and the home directory is set to the current directory. As a part of the pre-configuration, the project has been set up and Kubeconfig is set up. The details that are shown on the terminal.

```
ibmcloud ce project current
theia@theiadocker-lavanyas:/home/project$ ibmcloud ce project current
Getting the current project context...
OK

Name:       Code Engine - sn-labs-lavanyas
ID:         ee5183a9-4516-4bd1-8f4e-4a8615cafd81
Subdomain:  v9oc2xsjxaz
Domain:     us-south.codeengine.appdomain.cloud
Region:     us-south

Kubernetes Config:
Context:              v9oc2xsjxaz
Environment Variable:  export KUBECONFIG="/home/theia/.bluemix/plugins/code-engine/Code Engine -
 sn-labs-lavanyas-ee5183a9-4516-4bd1-8f4e-4a8615cafd81.yaml"
theia@theiadocker-lavanyas:/home/project$
```

Deploy Sentiment Analysis on Code Engine as a Microservice

- In the code engine CLI, change to `server/djangoapp/microservices` directory.

```bash
cd xrwvm-fullstack_developer_capstone/server/djangoap
```

Run

You have been provided with sentiment_analyzer.py which uses NLTK for sentiment analysis. You are also provided with a Dockerfile which you will use to deploy this service in Code Engine and consume it as a microservice. Take a look at these files.

- Run the command given below to docker build the sentiment analyzer app

> *Note: The code engine instance is transient and is attached to your lab space username.*

```plaintext
docker build . -t us.icr.io/${SN_ICR_NAMESPACE}/senti_analy
```

- Push the docker image by running the command given below.

```plaintext
docker push us.icr.io/${SN_ICR_NAMESPACE}/senti_analyzer
```

- Deploy the senti_analyzer application on code engine.

```
ibmcloud ce application create --name sentianalyzer --image
```

- Connect to the URL that is generated to access the microservices and check if the deployment is successful.
- If the application deployment verification was successful, attach `/analyze/Fantastic services` to the URL in the browser to see if it returns **positive**.

Assessment:

For Option 1: AI-Graded Submission and Evaluation: Run the command below in a new terminal. Copy and paste the terminal output along with the command and save it in a text file named `analyzereview` for the final project submission and evaluation.

```
curl -X GET "<Your-Sentimental-analyzer-URL>/analyze/Fantas
```

> *Note: Replace the Your-Sentimental-analyzer-URL with the URL that is generated after deployment.*

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of the sentiment along with the URL as shown below and save it as `sentiment_analyzer.png` or `sentiment_analyzer.jpeg`.

- Open `djangoapp/.env` and replace `your code engine deployment url` with the deployment URL you obtained above.

> *It is essential to include the `/` at the end of the URL. Ensure that it is copied.*

plaintext

```
sentiment_analyzer_url=your code engine deployment url
```

- Update `djangoapp/restapis.py` and add the function given below in it to consume the microservice to analyze sentiments.

python

```python
def analyze_review_sentiments(text):
    request_url = sentiment_analyzer_url+"analyze/"+text
    try:
        # Call get method of requests library with URL and
        response = requests.get(request_url)
        return response.json()
    except Exception as err:
        print(f"Unexpected {err=}, {type(err)=}")
        print("Network exception occurred")
```

Create Django Views to Get Dealers

- Update the `get_dealerships` view method in `djangoapp/views.py` with the code given below. It will use the `get_request` you implemented in the `restapis.py` passing the `/fetchDealers` endpoint.

python

```python
#Update the `get_dealerships` render list of dealerships al
def get_dealerships(request, state="All"):
    if(state == "All"):
        endpoint = "/fetchDealers"
    else:
        endpoint = "/fetchDealers/"+state
    dealerships = get_request(endpoint)
    return JsonResponse({"status":200,"dealers":dealerships
```

- Configure the route for `get_dealerships` view method in `url.py`:

python

```python
path(route='get_dealers', view=views.get_dealerships, name=
    path(route='get_dealers/<str:state>', view=views.get_de
```

- Create a `get_dealer_details` method which takes the dealer_id as a parameter in `views.py` and add a mapping urls.py. It will use the `get_request` you implemented in the `restapis.py` passing the `/fetchDealer/<dealer id>` endpoint.

▶ Click here for a sample

parameter in `views.py` and add a mapping urls.py. It will use the `get_request` you implemented in the `restapis.py` passing the `/fetchReviews/dealer/<dealer id>` endpoint. It will also call `analyze_review_sentiments` in `restapis.py` to consume the microservice and determine the sentiment of each of the reviews and set the value in the `review_detail` dictonary which is returned as a JsonResponse.

The value of `sentiment` attribute will be determined by sentiment analysis microservice. It could be `positive`, `neutral`, or `negative`.

▶ Click here for sample

Create a Django View to Post a Dealer Review

By now you have learned how to make various GET calls.

- Open `restapis.py`, add a `post_review` method which will take a data dictionary in and call the `add_review` in the backend. The dictionary would take all the values required for the dealership review as key-value, pair.

```python
def post_review(data_dict):
    request_url = backend_url+"/insert_review"
    try:
        response = requests.post(request_url,json=data_dict
        print(response.json())
        return response.json()
    except:
        print("Network exception occurred")
```

- Open `views.py`, create a new `def add_review(request):` method to handle review post request. In the `add_review` view method:
    - First check if user is authenticated because only authenticated users can post reviews for a dealer.
    - Call the `post_request` method with the dictionary
    - Return the result of `post_request` to `add_review` view method. You may print the post response
    - Return a success status and message as JSON
    - Configure the route for `add_review` view in `url.py`.

```python
def add_review(request):
    if(request.user.is_anonymous == False):
        data = json.loads(request.body)
        try:
            response = post_review(data)
            return JsonResponse({"status":200})
        except:
            return JsonResponse({"status":401,"message":"Er
    else:
        return JsonResponse({"status":403,"message":"Unauth
```

```python
    path(route='add_review', view=views.add_review, name='a
```

- Import the methods from `restapis.py` for use inside `views.py`.

```py
from .restapis import get_request, analyze_review_sentiment
```

Commit your Updated Project to GitHub

Commit all updates in the GitHub repository you created so that you can save your work.

If you need to refresh your memory on how to commit and push to GitHub in Theia lab environment, refer to the lab Working with git in the Theia lab environment

External References

- Requests Developer Interface
- NLTK

Summary

Congratulations on completing the **Lab: Create Django Proxy Services Of Backend APIs!**

In this lab, you have learned how to create proxy services to call the cloud functions in Django, convert their JSON results into Python objects such as `CarDealer` or `DealerReview`, and return the objects as a HTTPResonse.

Author(s)

Lavanya

Yan Luo

Other Contributor(s)

Upkar Lidder

Commit your Updated Project to GitHub

Commit all updates in the GitHub repository you created so that you can save your work.

If you need to refresh your memory on how to commit and push to

Theia lab environment

← Create a Django View to ...

External References →